

DL_03_arquitecturas

March 13, 2026

1 Arquitecturas de Deep Learning

1.1 Contenido

1. Redes Fully Connected (MLP)
2. Redes Convolucionales (CNN)
3. Redes Recurrentes (RNN/LSTM)
4. Autoencoders
5. Transfer Learning
6. Comparativa y Selección

```
[1]: import torch
import torch.nn as nn
import torch.nn.functional as F
import matplotlib.pyplot as plt
import numpy as np
```

1.2 1. Redes Fully Connected (MLP)

El **Perceptrón Multicapa** conecta cada neurona con todas las de la capa siguiente.

1.2.1 Cuándo usar MLP:

- Datos tabulares (features estructuradas)
- Clasificación/regresión con features ya extraídas
- Como capas finales de otras arquitecturas

1.2.2 Limitaciones:

- No captura relaciones espaciales (imágenes)
- No captura relaciones temporales (secuencias)
- Muchos parámetros para datos de alta dimensión

```
[2]: class MLP(nn.Module):
    """
    Perceptrón Multicapa configurable.

    Arquitectura: Input → [Linear → BN → ReLU → Dropout] × N → Output
```

```

"""
def __init__(self, input_size, hidden_sizes, output_size, dropout=0.2):
    super().__init__()

    layers = []
    prev_size = input_size

    for hidden_size in hidden_sizes:
        layers.extend([
            nn.Linear(prev_size, hidden_size),
            nn.BatchNorm1d(hidden_size),
            nn.ReLU(),
            nn.Dropout(dropout)
        ])
        prev_size = hidden_size

    layers.append(nn.Linear(prev_size, output_size))
    self.network = nn.Sequential(*layers)

    def forward(self, x):
        if x.dim() > 2:
            x = x.view(x.size(0), -1)  # Flatten
        return self.network(x)

# Ejemplo
mlp = MLP(input_size=784, hidden_sizes=[256, 128], output_size=10)
x = torch.randn(32, 784)
out = mlp(x)
print(f"MLP: {x.shape} → {out.shape}")
print(f"Parámetros: {sum(p.numel() for p in mlp.parameters()):,}")

```

MLP: torch.Size([32, 784]) → torch.Size([32, 10])

Parámetros: 235,914

1.3 2. Redes Convolucionales (CNN)

Las CNNs explotan la estructura espacial de las imágenes mediante filtros locales que se deslizan por la imagen.

1.3.1 Componentes clave:

1. **Convolución:** Detecta patrones locales (bordes, texturas)
2. **Pooling:** Reduce dimensiones, aporta invarianza
3. **Stride/Padding:** Controla tamaño de salida

1.3.2 Ventajas:

- **Compartición de pesos:** Mismo filtro en toda la imagen
- **Conectividad local:** Cada neurona ve solo una región

- **Jerarquía de features:** Capas profundas = features abstractas

```
[3]: # Visualizar operación de convolución
print("=== OPERACIÓN DE CONVOLUCIÓN ===")

# Imagen 5x5, 1 canal
imagen = torch.tensor([
    [0, 0, 0, 0, 0],
    [0, 1, 1, 1, 0],
    [0, 1, 0, 1, 0],
    [0, 1, 1, 1, 0],
    [0, 0, 0, 0, 0]
], dtype=torch.float32).unsqueeze(0).unsqueeze(0) # [1, 1, 5, 5]

# Filtro 3x3 (detector de bordes verticales)
filtro = torch.tensor([
    [-1, 0, 1],
    [-1, 0, 1],
    [-1, 0, 1]
], dtype=torch.float32).unsqueeze(0).unsqueeze(0) # [1, 1, 3, 3]

# Aplicar convolución
output = F.conv2d(imagen, filtro, padding=0)

fig, axes = plt.subplots(1, 3, figsize=(12, 4))

axes[0].imshow(imagen.squeeze(), cmap='gray')
axes[0].set_title('Imagen 5x5')

axes[1].imshow(filtro.squeeze(), cmap='RdBu', vmin=-1, vmax=1)
axes[1].set_title('Filtro 3x3\n(detector bordes)')

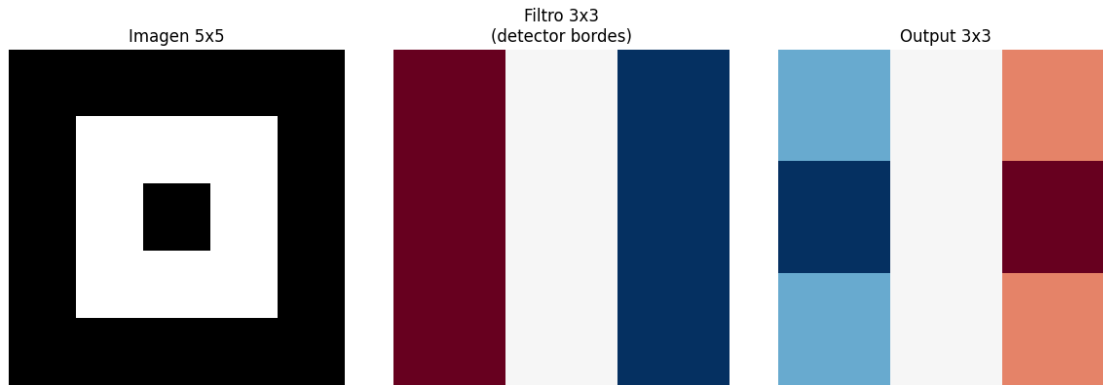
axes[2].imshow(output.squeeze().detach(), cmap='RdBu')
axes[2].set_title('Output 3x3')

for ax in axes:
    ax.axis('off')

plt.tight_layout()
plt.show()

print(f"Input: {imagen.shape} → Output: {output.shape}")
```

=== OPERACIÓN DE CONVOLUCIÓN ===



Input: torch.Size([1, 1, 5, 5]) → Output: torch.Size([1, 1, 3, 3])

```
[4]: class CNN(nn.Module):
    """
    Red Convolutional para clasificación de imágenes.

    Arquitectura típica:
    [Conv → BN → ReLU → Conv → BN → ReLU → MaxPool] × N → Flatten → FC
    """
    def __init__(self, in_channels=3, num_classes=10):
        super().__init__()

        # Bloque 1: 32x32 → 16x16
        self.block1 = nn.Sequential(
            nn.Conv2d(in_channels, 32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(),
            nn.Conv2d(32, 32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(),
            nn.MaxPool2d(2, 2)
        )

        # Bloque 2: 16x16 → 8x8
        self.block2 = nn.Sequential(
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(),
            nn.Conv2d(64, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(),
            nn.MaxPool2d(2, 2)
        )
```

```

# Bloque 3: 8x8 → 4x4
self.block3 = nn.Sequential(
    nn.Conv2d(64, 128, kernel_size=3, padding=1),
    nn.BatchNorm2d(128),
    nn.ReLU(),
    nn.MaxPool2d(2, 2)
)

# Clasificador
self.classifier = nn.Sequential(
    nn.Flatten(),
    nn.Linear(128 * 4 * 4, 256),
    nn.ReLU(),
    nn.Dropout(0.5),
    nn.Linear(256, num_classes)
)

def forward(self, x):
    x = self.block1(x)
    x = self.block2(x)
    x = self.block3(x)
    x = self.classifier(x)
    return x

# Ejemplo
cnn = CNN(in_channels=3, num_classes=10)
x = torch.randn(32, 3, 32, 32) # Batch de 32 imágenes RGB 32x32
out = cnn(x)
print(f"CNN: {x.shape} → {out.shape}")
print(f"Parámetros: {sum(p.numel() for p in cnn.parameters()):,}")

```

CNN: torch.Size([32, 3, 32, 32]) → torch.Size([32, 10])

Parámetros: 667,178

1.3.3 Cálculo de tamaños en CNN

Para una capa convolucional:

$$H_{out} = \frac{H_{in} + 2 \times padding - kernel_size}{stride} + 1$$

Con padding=1, kernel=3, stride=1: el tamaño se mantiene.

Con MaxPool2d(2,2): el tamaño se divide por 2.

1.4 3. Redes Recurrentes (RNN/LSTM)

Las **RNNs** procesan secuencias manteniendo un estado oculto que captura información de pasos anteriores.

1.4.1 Problema de RNN vanilla: Vanishing Gradient

Los gradientes se multiplican en cada paso temporal, tendiendo a 0 (vanishing) o infinito (exploding).

1.4.2 LSTM (Long Short-Term Memory)

Resuelve el problema con **gates** que controlan el flujo de información: - **Forget gate**: Qué olvidar del estado anterior - **Input gate**: Qué información nueva añadir - **Output gate**: Qué parte del estado emitir

```
[5]: class LSTMClassifier(nn.Module):
    """
    LSTM para clasificación de secuencias (ej: análisis de sentimientos).

    Arquitectura: Embedding → LSTM bidireccional → FC
    """
    def __init__(self, vocab_size, embedding_dim, hidden_dim, output_dim,
                  n_layers=2, bidirectional=True, dropout=0.3):
        super().__init__()

        self.embedding = nn.Embedding(vocab_size, embedding_dim, padding_idx=0)

        self.lstm = nn.LSTM(
            input_size=embedding_dim,
            hidden_size=hidden_dim,
            num_layers=n_layers,
            batch_first=True,
            bidirectional=bidirectional,
            dropout=dropout if n_layers > 1 else 0
        )

        # Bidireccional duplica el hidden_dim
        self.fc = nn.Linear(hidden_dim * 2 if bidirectional else hidden_dim,
                             ↪output_dim)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        # x: (batch, seq_len) - índices de palabras

        # Embedding
        embedded = self.embedding(x) # (batch, seq_len, embedding_dim)

        # LSTM
```

```

        output, (hidden, cell) = self.lstm(embedded)
        # hidden: (n_layers * n_directions, batch, hidden_dim)

        # Concatenar últimos hidden states (forward y backward)
        hidden = torch.cat((hidden[-2,:,:], hidden[-1,:,:]), dim=1)

        # Clasificación
        hidden = self.dropout(hidden)
        return self.fc(hidden)

# Ejemplo
lstm = LSTMClassifier(vocab_size=10000, embedding_dim=128, hidden_dim=256,
    ↪output_dim=2)
x = torch.randint(0, 10000, (32, 100)) # 32 secuencias de 100 tokens
out = lstm(x)
print(f"LSTM: {x.shape} → {out.shape}")
print(f"Parámetros: {sum(p.numel() for p in lstm.parameters()):,}")

```

LSTM: torch.Size([32, 100]) → torch.Size([32, 2])
 Parámetros: 3,648,514

```

[6]: class LSTMTimeSeries(nn.Module):
    """
    LSTM para predicción de series temporales.

    Input: secuencia de valores numéricos
    Output: siguiente valor(es) predicho(s)
    """
    def __init__(self, input_dim, hidden_dim, output_dim, n_layers=2):
        super().__init__()

        self.lstm = nn.LSTM(
            input_size=input_dim,
            hidden_size=hidden_dim,
            num_layers=n_layers,
            batch_first=True,
            dropout=0.2
        )

        self.fc = nn.Linear(hidden_dim, output_dim)

    def forward(self, x):
        # x: (batch, seq_len, input_dim)
        output, (hidden, cell) = self.lstm(x)

        # Usar último output
        return self.fc(output[:, -1, :])

```

```
# Ejemplo: predecir siguiente valor de 5 features
lstm_ts = LSTMTimeSeries(input_dim=5, hidden_dim=64, output_dim=1)
x = torch.randn(32, 50, 5) # 32 series, 50 timesteps, 5 features
out = lstm_ts(x)
print(f"LSTM TimeSeries: {x.shape} → {out.shape}")
```

LSTM TimeSeries: torch.Size([32, 50, 5]) → torch.Size([32, 1])

1.5 4. Autoencoders

Los **autoencoders** aprenden a comprimir y reconstruir datos, útiles para: - Reducción de dimensionalidad - Detección de anomalías - Generación de datos (VAE)

```
[7]: class Autoencoder(nn.Module):
    """
    Autoencoder básico.

    Arquitectura:
    Encoder: Input → compresión → Latent (bottleneck)
    Decoder: Latent → expansión → Reconstrucción
    """
    def __init__(self, input_dim, latent_dim):
        super().__init__()

        # Encoder
        self.encoder = nn.Sequential(
            nn.Linear(input_dim, 256),
            nn.ReLU(),
            nn.Linear(256, 128),
            nn.ReLU(),
            nn.Linear(128, latent_dim)
        )

        # Decoder (espejo del encoder)
        self.decoder = nn.Sequential(
            nn.Linear(latent_dim, 128),
            nn.ReLU(),
            nn.Linear(128, 256),
            nn.ReLU(),
            nn.Linear(256, input_dim),
            nn.Sigmoid() # Output entre 0 y 1
        )

    def encode(self, x):
        return self.encoder(x)

    def decode(self, z):
```



```

        return self.decoder(z)

    def forward(self, x):
        z = self.encode(x)
        return self.decode(z)

# Ejemplo: comprimir MNIST (784 → 32)
ae = Autoencoder(input_dim=784, latent_dim=32)
x = torch.randn(32, 784)
reconstructed = ae(x)
latent = ae.encode(x)
print(f"Autoencoder: {x.shape} → latent {latent.shape} → {reconstructed.shape}")

```

Autoencoder: torch.Size([32, 784]) → latent torch.Size([32, 32]) → torch.Size([32, 784])

```

[8]: class VAE(nn.Module):
    """
    Variational Autoencoder.

    Diferencia con AE: el espacio latente es una distribución (no un punto).
    Permite generar nuevas muestras.
    """
    def __init__(self, input_dim, latent_dim):
        super().__init__()

        # Encoder
        self.encoder = nn.Sequential(
            nn.Linear(input_dim, 256),
            nn.ReLU(),
            nn.Linear(256, 128),
            nn.ReLU()
        )

        # Capas para media y varianza
        self.fc_mu = nn.Linear(128, latent_dim)
        self.fc_var = nn.Linear(128, latent_dim)

        # Decoder
        self.decoder = nn.Sequential(
            nn.Linear(latent_dim, 128),
            nn.ReLU(),
            nn.Linear(128, 256),
            nn.ReLU(),
            nn.Linear(256, input_dim),
            nn.Sigmoid()
        )

```

```

def encode(self, x):
    h = self.encoder(x)
    return self.fc_mu(h), self.fc_var(h)

def reparameterize(self, mu, log_var):
    """Reparametrization trick:  $z = \mu + \sigma * \epsilon$ """
    std = torch.exp(0.5 * log_var)
    eps = torch.randn_like(std)
    return mu + eps * std

def decode(self, z):
    return self.decoder(z)

def forward(self, x):
    mu, log_var = self.encode(x)
    z = self.reparameterize(mu, log_var)
    return self.decode(z), mu, log_var

def sample(self, num_samples, device='cpu'):
    """Genera nuevas muestras del espacio latente."""
    z = torch.randn(num_samples, self.fc_mu.out_features).to(device)
    return self.decode(z)

# Ejemplo
vae = VAE(input_dim=784, latent_dim=32)
x = torch.randn(32, 784)
recon, mu, log_var = vae(x)
print(f"VAE: {x.shape} → mu {mu.shape}, recon {recon.shape}")

```

VAE: torch.Size([32, 784]) → mu torch.Size([32, 32]), recon torch.Size([32, 784])

1.6 5. Transfer Learning

Transfer Learning usa modelos preentrenados en grandes datasets (ImageNet) y los adapta a tu tarea específica.

1.6.1 Estrategias:

1. **Feature Extraction:** Congelar backbone, entrenar solo clasificador
2. **Fine-tuning:** Entrenar todo con learning rate bajo
3. **Gradual unfreezing:** Descongelar capas progresivamente

```

[9]: import torchvision.models as models

# Cargar modelo preentrenado
resnet = models.resnet18(weights='IMAGENET1K_V1')

```

```

print("ResNet18 cargado con pesos de ImageNet")

# Ver estructura
print(f"\nÚltima capa original: {resnet.fc}")
print(f"Clases originales: 1000 (ImageNet)")

```

Downloading: "https://download.pytorch.org/models/resnet18-f37072fd.pth" to
/home/jeironpro/.cache/torch/hub/checkpoints/resnet18-f37072fd.pth

100%| | 44.7M/44.7M [00:01<00:00, 28.6MB/s]

ResNet18 cargado con pesos de ImageNet

Última capa original: Linear(in_features=512, out_features=1000, bias=True)
Clases originales: 1000 (ImageNet)

```

[10]: # Estrategia 1: Feature Extraction
      # Congelar todas las capas excepto la última

resnet_fe = models.resnet18(weights='IMAGENET1K_V1')

# Congelar backbone
for param in resnet_fe.parameters():
    param.requires_grad = False

# Reemplazar última capa
num_features = resnet_fe.fc.in_features
resnet_fe.fc = nn.Sequential(
    nn.Linear(num_features, 256),
    nn.ReLU(),
    nn.Dropout(0.3),
    nn.Linear(256, 10) # 10 clases para tu tarea
)

# Contar parámetros entrenables
trainable = sum(p.numel() for p in resnet_fe.parameters() if p.requires_grad)
total = sum(p.numel() for p in resnet_fe.parameters())
print(f"Feature Extraction:")
print(f"  Entrenables: {trainable:,} / {total:,} ({100*trainable/total:.1f}%)")

```

Feature Extraction:
Entrenables: 133,898 / 11,310,410 (1.2%)

```

[11]: # Estrategia 2: Fine-tuning completo
      # Entrenar todo con learning rate bajo

resnet_ft = models.resnet18(weights='IMAGENET1K_V1')

# Reemplazar última capa

```

```

num_features = resnet_ft.fc.in_features
resnet_ft.fc = nn.Linear(num_features, 10)

# Todo entrenable, pero con learning rates diferentes
optimizer = torch.optim.Adam([
    {'params': resnet_ft.layer4.parameters(), 'lr': 1e-4}, # Capas altas: lr_
    ↪bajo
    {'params': resnet_ft.fc.parameters(), 'lr': 1e-3}      # Clasificador: lr_
    ↪normal
], lr=1e-5) # Default para el resto

trainable = sum(p.numel() for p in resnet_ft.parameters() if p.requires_grad)
print(f"Fine-tuning:")
print(f"  Entrenables: {trainable:,} (100%)")

```

Fine-tuning:

Entrenables: 11,181,642 (100%)

1.7 6. Comparativa y Selección de Arquitecturas

Tipo de Dato	Arquitectura Recomendada
Tabular (features numéricas)	MLP, XGBoost
Imágenes	CNN (ResNet, EfficientNet)
Texto	LSTM, Transformer (BERT)
Secuencias numéricas	LSTM, 1D-CNN
Grafos	GNN (GraphSAGE, GAT)
Audio	1D-CNN, Transformer

1.7.1 Factores de decisión:

1. Tamaño del dataset

- Pequeño (< 10k): Transfer learning, modelos simples
- Grande (> 100k): Entrenar desde cero posible

2. Recursos computacionales

- Limitados: Modelos pequeños, MobileNet
- GPU potente: ResNet-50+, Transformers

3. Latencia requerida

- Tiempo real: MobileNet, SqueezeNet
- Batch: Modelos más grandes

```

[12]: # Resumen de parámetros por arquitectura
print("=== COMPARATIVA DE ARQUITECTURAS ===")
print(f"{'Modelo':<25} {'Parámetros':>15} {'Uso típico'}")
print("-" * 60)

modelos = [
    ("MLP (784→256→128→10)", MLP(784, [256, 128], 10), "Datos tabulares"),

```

```

("CNN simple (CIFAR)", CNN(3, 10), "Imágenes pequeñas"),
("LSTM (texto)", LSTMClassifier(10000, 128, 256, 2), "NLP"),
("Autoencoder (784+32)", Autoencoder(784, 32), "Compresión"),
("ResNet-18", models.resnet18(), "Imágenes (transfer)"),
]

for nombre, modelo, uso in modelos:
    params = sum(p.numel() for p in modelo.parameters())
    print(f"{nombre:<25} {params:>15,} {uso}")

```

=== COMPARATIVA DE ARQUITECTURAS ===

Modelo	Parámetros	Uso típico
MLP (784+256+128+10)	235,914	Datos tabulares
CNN simple (CIFAR)	667,178	Imágenes pequeñas
LSTM (texto)	3,648,514	NLP
Autoencoder (784+32)	476,720	Compresión
ResNet-18	11,689,512	Imágenes (transfer)

1.8 Resumen

Arquitectura	Fortaleza	Input típico
MLP	Simplicidad	Vectores de features
CNN	Patrones espaciales	Imágenes ($H \times W \times C$)
RNN/LSTM	Patrones temporales	Secuencias ($T \times D$)
Autoencoder	Compresión/Generación	Cualquier dato
Transformer	Atención, paralelismo	Secuencias largas

Siguiente: [DL_04_entrenamiento.ipynb](#)